

Passive circuit Package Description. Version 0.1

February 1, 2026

1 Introduction

Passive circuit package (`passivecircuit.py`, depends on `lingencthy.py`) is intended for simulation of linear circuits at finite frequency. It implements a traditional Kirchhoff circuit theory. The circuit consists of nodes connected to the terminals and to each other. The connectors between terminals/nodes are two-pole conductors, capacitors, inductors, and four-pole transformers. Most important output computed is the conductance (more exactly, admittance) matrix in the space of the terminals. There are also tools to assert electric currents in terminals and voltages in the nodes, given the voltages at the terminals, and the impedance matrix of the circuit (the latter requires grounding of a terminal)

The actual use of the package starts with creating an instance of the **PassiveCircuit** class. Then the circuit is *built* by adding the terminals, nodes, and the connectors. It is so that each element of the circuit should be given a name, and addressed by this name. If you do not like giving names, make your own script to do this for you.

The computation for a circuit involves inversion of a matrix with a dimension given by the total number of nodes. This is fast for the circuits of a reasonable size (≤ 10 elements).

2 Building a circuit

Import **PassiveCircuit** from the package `passivecircuits`. The instance of the class, our circuit, is created by calling

PassiveCircuit ()

This returns the class instance. There are no arguments.

The following member functions of the instance add the elements of the setup without connecting them:

addTerminal(name)

Adds a terminal with a given name.

addNormalNode(name):

Adds a node of a given name.

The following functions add the connections.

addConductor(name,code,name1,name2,conductance=1.0):

This adds a conductor of a given name. The **code** specifies if it connects nodes or terminals. The possible values of **code** are 'TT' (connects two terminals), 'NT' (connects a node and a terminal), 'NN' (connects two nodes). The arguments **name1**, **name2** are the names of these nodes/terminals that must already be added to the circuit. The conductance may be specified right away: must be positive.

addCapacitor(name,code,name1,name2,capacitance=1.0):

This adds a capacitor of a given name. The meaning of the arguments is the same as for **addConductor**()

addCapacitor(name,code,name1,name2,capacitance=1.0):

This adds a capacitor of a given name. The meaning of the arguments is the same as for **addConductor**()

addInductor(name,code,name1,name2,inductance=1.0):

This adds an inductor of a given name. The meaning of the arguments is the same as for **addConductor**()

addTransformer (name, code ,name1, name2, name3, name4,

La=1.0,Lb=1.0,Ra=0,Rb=0,alpha=0.999,polarity=1.0)

This adds a transformer of a given name. It is a four-pole element connecting nodes/terminals **name1**, **name2**, **name3**, **name4**. The string **code** is made of the letters N and T indicating if **name1-4** is a node or a terminal. The meaning of the parameters is as follows: the transformer consists of two isolated coils *a* (connected to **name1**, **name2**) and *b* (connected to **name2**, **name3**). *La*, *Lb* are the inductances of the coils, *Ra*, *Rb* are (very small) residual resistances of the coils. **polarity** can take values ± 1 , and **alpha** in the interval $(-1, 1)$ such that the transinductance reads

$$L_{\text{trans}} = \sqrt{LaLb} \times \text{polarity} \times \text{alpha} \quad (1)$$

The ideal transformer corresponds to **alpha** = ± 1 . If the transformer is close to ideal, one has to choose the parameters with care. The point is that the admittance of the transformer is inversely proportional to

$$D \equiv -L_a L_b (1 - \alpha^2) \omega^2 + j\omega(L_a R_b + L_b R_a) + R_a R_b. \quad (2)$$

and the admittance changes from inductive to resistive response at frequencies $\omega \simeq (R/L)/(1 - \alpha^2)$, that can be high if the transformer is close to ideal one. It is therefore not allowed to set **alpha** to ± 1 and any of *R_{a,b}* to zero.

The following connectors will make the circuit asymmetric with respect to time reversal.

addHallTripole(name,code,name1,name2,name3, conductance=1.0):

addHallFourpole(name,code,name1,name2,name3, name4, conductance=1.0):

Those represent ideal symmetric Hall elements with 3 or 4 poles and given con-

ductance g such that the current to terminal/node i is given by $I_i = g(V_{i-1} - V_i)$. To make a non-ideal Hall element, add conductors to the poles.

Importantly, the circuit should be *finalized* after adding all elements.

finalize ()

The build-up cannot be changed after that, but the parameters of the existing elements can be changed or set anew.

3 Setting and changing the parameters

This is accomplished by the following functions:

setOmega (omega):

This sets ω for all elements of the circuit and simultaneously computes conductance matrix. Zero ω is not allowed.

If you change the element parameters with the functions below, and do not change the frequency, do not forget to call **compute**() to recompute the conductance matrix.

setConductor(name, conductance)

Sets the conductance of the named connector.

setCapacitor(name, capacitance)

Sets the capacitance of the named capacitor.

setInductance(name, inductance)

Sets the inductance of the named inductor.

setTransformer(name, parameter=newvalue, ...)

Sets the parameters of the named transformer, see **addTransformer**() Only changes the named parameters, the rest remain as before. For instance, **setCapacitor**('FirstTrans', alpha=0.5, La=10.0) changes only alpha and La of the transformer 'FirstTrans'.

setHallTripole(name, conductance)

setHallFourpole(name, conductance)

Sets the conductance of the named Hall element.

4 Getting the results

The matrix **ConductanceMatrix** in principle contains all results for given settings of element parameters and frequency. If N_t is the number of terminals, this is a $N_t \times N_t$ matrix that relates the N_t -dimensional vector of currents in the terminals to N_t -dimensional vector of voltages.

For convenience, there are helper functions for elements this matrix.

CMel(name1,name2):

returns the element of the conductance matrix between the named terminals.

There is also a set of convenience tools to inspect currents and voltages in the nodes at a given setting of voltages in all terminals. The default setting is the unit voltage at the first defined terminal, and zero voltage at the rest. This setting can be changed by

setVoltage(name,value): This sets the named terminal to the voltage given by **value**. The function also works with the lists of names and values, to simplify the setting of big number of terminals.

To come down to currents one calls

computeCurrents(): This computes and stores the result in **Currents** The following helper function returns the current in the named terminal

current(name):

Similarly, to access the voltages in the nodes, one calls

computeNodeVoltages():

After this, the helper function

nodevoltage(name):

returns the voltage in the named node

Finally, the resistance matrix can be computed and returned by

resistanceMatrix(ground = name):

For this, you should ground one of the terminals added, since the dimension of the resistance matrix is $(N_t - 1) \times (N_t - 1)$ If you have defined the terminal 'Ground' already, you may skip the argument.

Check the example file *check-passivecircuit.py* for examples of building and use. Happy computing!